

Assignment 5: Quicksort Algorithm: Implementation, Analysis, and Randomization

Haeri Kyoung

University of the Cumberland

MSCS-532-M20 – Algorithms and Data Structures

Professor Brandon Bass

June 27, 2025

Implementation Overview

This assignment involved implementing two versions of the Quicksort algorithm and analyzing their performance both theoretically and empirically. The two versions include a standard deterministic implementation and a randomized version where the pivot is chosen randomly at each recursive step. I also measured and compared their behavior on datasets of different sizes and distributions to understand how randomization impacts the algorithm in practice.

Time Complexity Analysis

In the deterministic version, I selected the last element in the array as the pivot. The algorithm partitions the input into two sublists based on the pivot, with values less than or equal to the pivot going to the left and those greater going to the right. This process repeats recursively for each sublist until the full array is sorted. However, when the input data is already sorted or reverse sorted, the partitions become highly unbalanced. In those cases, each recurs...

In contrast, the randomized version selects a pivot randomly from the current subarray. This breaks predictable patterns and results in more balanced partitions. Although the worst-case time complexity is still order n^2 , the probability of encountering that scenario is significantly reduced, making randomized Quicksort more robust in practical use.

According to Cormen et al. (2009), the best and average-case time complexity for Quicksort is order $n \log n$. Each recursive level processes all elements once, and balanced partitions lead to $\log n$ levels. Therefore, the total number of operations is proportional to $n \log n$.

Space Complexity

Both versions of Quicksort use recursion, which contributes to the space complexity. In the best and average cases, the recursive depth is $\log n$, resulting in $\log n$ stack frames. In the worst case,

where the recursion is deeply unbalanced, the stack can grow to n levels, resulting in linear space usage. Neither version uses additional memory beyond the recursion stack.

Empirical Results

To support the theoretical analysis, I tested both algorithms on arrays of various sizes and distributions, including random, sorted, and reverse sorted inputs. The deterministic version performed well on random data but slowed considerably on sorted and reverse sorted inputs. The randomized version, on the other hand, maintained stable performance across all cases, highlighting its resilience to unfavorable input patterns.

Conclusion

This assignment demonstrated how a small design decision like pivot selection can significantly impact algorithm performance. Randomized Quicksort consistently delivered better results and avoided performance degradation in worst-case scenarios. Understanding both theoretical and empirical behaviors of algorithms helps in selecting the right approach for different practical situations.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to algorithms (3rd ed.). MIT Press.